

Artificial Potential Field and Extended Kalman Filter Methods

Equations and Source-Code Mapping

Contents

Abbreviations and full forms	3
1 Six-state vessel EKF	3
1.1 State vector and motion model	3
1.2 State-transition Jacobian	4
1.3 Prediction step	5
1.4 Pose and yaw-rate measurement models	5
1.5 Measurement update	6
2 Five-state obstacle CTRV-EKF	6
2.1 State vector	7
2.2 CTRV prediction	7
2.3 Numerical Jacobian	7
2.4 Process noise	8
2.5 Position measurement update	8
2.6 Future position and trajectory	9
3 APF obstacle geometry	9
3.1 Elliptical level set	9
3.2 Smooth elliptical repulsion	10
3.3 Observed dimensions in snapshot generation	10
4 APF attractive terms	11
4.1 Goal attraction	11
4.2 Path attraction	11
5 CPA-based dynamic-obstacle risk	12
6 APF force composition and control output	13
6.1 Total force	13

6.2	Force direction and yaw-rate command	14
7	Processing chain	14
8	Implementation index	15

Abbreviations and full forms

Symbol or term	Meaning
N, E	Northing and easting coordinates.
G or ϕ	Vessel heading or obstacle motion direction, in radians.
Δt	Discrete time step.
μ or $\mathbf{x}$	State estimate.
Σ or $\mathbf{P}$	State covariance matrix.
$\mathbf{Q}, \mathbf{R}$	Process-noise and measurement-noise covariance matrices.
$\mathbf{F}, \mathbf{H}$	Jacobians of the state-transition and measurement functions.
z	Sensor measurement.
$\mathbf{K}$	Kalman gain.
Body frame	Vessel-fixed frame, with x forward and y to starboard.
Earth/NE frame	Earth-fixed north-east frame.
APF	Artificial Potential Field.
COLREGs	International Regulations for Preventing Collisions at Sea.
CPA	Closest point of approach.
CTRV	Constant Turn Rate and Velocity.
DBSCAN	Density-Based Spatial Clustering of Applications with Noise.
TCPA, DCPA	Time to CPA and distance at CPA.
EKF	Extended Kalman Filter.
IMU	Inertial Measurement Unit.
LiDAR	Light Detection and Ranging.
PC	Principal Component.
PC1	First Principal Component.
PC2	Second Principal Component.
PCA	Principal Component Analysis.

Angular differences are normalised to $[-\pi, \pi)$ by `wrap_angle()`.

1 Six-state vessel EKF

In this chapter, N and E are northing and easting, G is the vessel heading, and dotted quantities are their time derivatives. Bold lower-case letters denote vectors and bold upper-case letters denote matrices. The subscripts b and e identify the body and earth frames, respectively; k is the discrete-time index. The symbols Δt , $\mathbf{u}$, $\mathbf{x}$, $\mathbf{p}$, $\mathbf{v}$, and $\mathbf{F}$ denote the sampling interval, thruster input, state, position, velocity, and force. The matrices $\mathbf{G}$, $\mathbf{R}_{be}(G)$, and $\mathbf{F}$ are, respectively, the thruster allocation matrix, the rotation from body to earth coordinates at heading G , and the state-transition Jacobian; the bold matrix $\mathbf{G}$ is distinguished from the scalar heading G by its typography. The functions f_{dyn} and f_{kin} are the vessel dynamics and rigid-body kinematics.

1.1 State vector and motion model

The vessel EKF uses the state vector

$$\mathbf{x} = \begin{bmatrix} N & E & G & \dot{N} & \dot{E} & \dot{G} \end{bmatrix}^T. \quad (1)$$

The thruster input is

$$\mathbf{u} = \begin{bmatrix} T_R & T_L \end{bmatrix}^T. \quad (2)$$

The allocation matrix maps the thruster forces to body-frame force and yaw moment,

$$\mathbf{F}_b = \mathbf{G}\mathbf{u}, \quad (3)$$

and the heading-dependent rotation maps this vector to the Earth-fixed frame,

$$\mathbf{F}_e = \mathbf{R}_{be}(G)\mathbf{F}_b. \quad (4)$$

The vessel dynamics produce the next Earth-frame velocity,

$$\mathbf{v}_{e,k+1} = f_{\text{dyn}}(\mathbf{v}_{e,k}, \mathbf{F}_e, \Delta t). \quad (5)$$

The Earth-frame velocity is transformed back to the body frame. Its surge and yaw-rate components form the kinematic input,

$$\mathbf{v} = v_b^x, \quad \omega = v_b^G. \quad (6)$$

The rigid-body kinematics then update the pose,

$$\mathbf{p}_{k+1} = f_{\text{kin}}(\mathbf{p}_k, [\mathbf{v}, \omega]^\top, \Delta t). \quad (7)$$

The implementation is located in `motion_model()`:

```
Fb = self.G @ control_input
H_eb = HomogeneousTransformation(state[N:E+1], state[G])
Fe = H_eb.H_R @ Fb

ve = self.robot.model(
    dynamics_translation_e,
    dynamics_rotation_e,
    Fe,
    state[DOTN:DOTG+1],
    dt,
)

vb = Inverse(H_eb.H_R) @ ve
u = Vector(2)
u[0, 0] = vb[0, 0]
u[1, 0] = vb[2, 0]

p = rigid_body_kinematics(state[N:G+1], u, dt)
p[2, 0] = p[2, 0] % (2 * np.pi)
```

1.2 State-transition Jacobian

The implementation uses a first-order discrete-time Jacobian. Here $\mathbf{F}_k$ is the Jacobian of the transition function at time k , and the unit entries preserve direct integration of northing, easting, and heading from their corresponding rates:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \quad (8)$$

```

F = Identity(6)
F[N, DOTN] = dt
F[E, DOTE] = dt
F[G, DOTG] = dt

```

1.3 Prediction step

The symbols $\hat{\mathbf{x}}_k$ and $\hat{\Sigma}_k$ denote the predicted state and covariance, while $\mathbf{x}_{k-1}$ and Σ_{k-1} are the previous corrected state and covariance. The transition function is f , $\mathbf{Q}_k$ is the process-noise covariance, and the superscript T denotes matrix transpose. The state and covariance predictions are

$$\hat{\mathbf{x}}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k, \Delta t), \quad (9)$$

$$\hat{\Sigma}_k = \mathbf{F}_k \Sigma_{k-1} \mathbf{F}_k^T + \mathbf{Q}_k. \quad (10)$$

```

def extended_kalman_filter_predict(mu, Sigma, u, f, Q, dt):
    pred_mu, F = f(mu, u, dt)
    pred_Sigma = F @ Sigma @ F.T + Q
    return pred_mu, pred_Sigma

```

1.4 Pose and yaw-rate measurement models

The measurement vector is $\mathbf{z}$, and h_{pose} is the pose measurement function. The matrix $\mathbf{H}$ is its measurement Jacobian. The scalar z_{gyro} is the measured yaw rate, and H_{gyro} is the corresponding Jacobian entry. The ArUco measurement contains northing, easting, and heading:

$$\mathbf{z}_{\text{pose}} = h_{\text{pose}}(\mathbf{x}) = \begin{bmatrix} N & E & G & 0 & 0 & 0 \end{bmatrix}^T. \quad (11)$$

Its Jacobian has unit entries at (N, N) , (E, E) , and (G, G) . The IMU model measures yaw rate:

$$z_{\text{gyro}} = \dot{G}, \quad H_{\text{gyro}}[\dot{G}, \dot{G}] = 1. \quad (12)$$

```

def h_pose_update(x):
    est_measurement = Vector(6)
    est_measurement[N] = x[N]
    est_measurement[E] = x[E]
    est_measurement[G] = x[G]
    H = Matrix(6, 6)
    H[N, N] = 1
    H[E, E] = 1
    H[G, G] = 1
    return est_measurement, H

def h_grate_update(x):
    est_measurement = Vector(6)
    est_measurement[DOTG] = x[DOTG]
    H = Matrix(6, 6)
    H[DOTG, DOTG] = 1
    return est_measurement, H

```

1.5 Measurement update

In this subsection, $\hat{z}$ is the predicted measurement, $\mathbf{y}$ is the innovation, $\mathbf{S}$ is the innovation covariance, and $\mathbf{K}$ is the Kalman gain. Superscripts $-$ and $+$ identify the predicted and corrected quantities. The matrix $\mathbf{I}$ is the identity matrix, and $\mathbf{R}$ is the measurement-noise covariance. The predicted measurement, innovation, innovation covariance, and Kalman gain are

$$\hat{z} = h(\hat{\mathbf{x}}), \quad (13)$$

$$\mathbf{y} = \mathbf{z} - \hat{z}, \quad (14)$$

$$\mathbf{S} = \mathbf{H}\Sigma^- \mathbf{H}^\top + \mathbf{R}, \quad (15)$$

$$\mathbf{K} = \Sigma^- \mathbf{H}^\top \mathbf{S}^{-1}. \quad (16)$$

The corrected state is

$$\mathbf{x}^+ = \mathbf{x}^- + \mathbf{K}\mathbf{y}. \quad (17)$$

The heading innovation is wrapped before correction:

$$y_G = \text{wrap}(z_G - \hat{z}_G). \quad (18)$$

The vessel EKF currently updates the covariance as

$$\Sigma^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\Sigma^-. \quad (19)$$

```
def extended_kalman_filter_update(
    mu, Sigma, z, h, R, wrap_index=None
):
    pred_z, H = h(mu)
    K = Sigma @ H.T @ Inverse(H @ Sigma @ H.T + R)
    delta_z = z - pred_z
    if wrap_index is not None:
        delta_z[wrap_index] = (
            delta_z[wrap_index] + np.pi
        ) % (2 * np.pi) - np.pi
    cor_mu = mu + K @ delta_z
    cor_Sigma = (Identity(mu.shape[0]) - K @ H) @ Sigma
    return cor_mu, cor_Sigma
```

In the main loop, the update order is thruster-based prediction, ArUco pose correction, and IMU yaw-rate correction.

2 Five-state obstacle CTRV-EKF

In this chapter, k is the discrete prediction index and Δt is the prediction interval. Obstacle position components are p_x and p_y , v is speed, ϕ is motion direction, and $\dot{\phi}$ is turn rate. A superscript meas marks a LiDAR measurement. Bold letters denote vectors or matrices; f is the CTRV state-transition function, $\mathbf{F}$ its Jacobian, $\mathbf{Q}$ the process-noise covariance, $\mathbf{R}$ the measurement-noise covariance, $\mathbf{H}$ the measurement Jacobian, $\mathbf{P}$ the state covariance, $\mathbf{K}$ the Kalman gain, and $\mathbf{I}$ the identity matrix. The symbols ϵ_i and $\mathbf{e}_i$ are the finite-difference step and the i th coordinate basis vector. The operator `diag` constructs a diagonal matrix, and τ is a future prediction horizon.

2.1 State vector

Each tracked obstacle uses

$$\mathbf{x} = \begin{bmatrix} p_x & p_y & v & \phi & \dot{\phi} \end{bmatrix}^T, \quad (20)$$

The five entries are ordered as two position coordinates, speed, motion direction, and turn rate, matching the state order used by the obstacle EKF.

2.2 CTRV prediction

When $|\dot{\phi}|$ exceeds the configured turn-rate threshold, the obstacle follows a circular-arc prediction. The subscript k denotes the current step and $k + 1$ the next step:

$$\phi_{k+1} = \phi_k + \dot{\phi}_k \Delta t, \quad (21)$$

$$p_{x,k+1} = p_{x,k} + \frac{v}{\dot{\phi}} [\sin(\phi_{k+1}) - \sin(\phi_k)], \quad (22)$$

$$p_{y,k+1} = p_{y,k} + \frac{v}{\dot{\phi}} [-\cos(\phi_{k+1}) + \cos(\phi_k)]. \quad (23)$$

When $|\dot{\phi}|$ is small, the model reduces to straight-line motion:

$$p_{x,k+1} = p_{x,k} + v \cos(\phi_k) \Delta t, \quad (24)$$

$$p_{y,k+1} = p_{y,k} + v \sin(\phi_k) \Delta t. \quad (25)$$

```
if abs(phi_dot) >= self.config.turn_rate_epsilon_rad_s:
    next_phi = wrap_angle(phi + phi_dot * dt)
    px_next = px + v / phi_dot * (
        np.sin(next_phi) - np.sin(phi)
    )
    py_next = py + v / phi_dot * (
        -np.cos(next_phi) + np.cos(phi)
    )
else:
    next_phi = (
        phi if v < self.config.heading_hold_speed_m_s
        else wrap_angle(phi)
    )
    px_next = px + v * np.cos(phi) * dt
    py_next = py + v * np.sin(phi) * dt
```

2.3 Numerical Jacobian

The quantity $\mathbf{F}_{:,i}$ is column i of the transition Jacobian, and $\epsilon_i \mathbf{e}_i$ perturbs only state coordinate i . The obstacle model uses a forward-difference Jacobian rather than a handwritten analytical Jacobian:

$$\mathbf{F}_{:,i} \approx \frac{f(\mathbf{x} + \epsilon_i \mathbf{e}_i, \Delta t) - f(\mathbf{x}, \Delta t)}{\epsilon_i}. \quad (26)$$

```
base = self._ctrv_step(state, dt)
for index, step in enumerate(steps):
    perturbed = state.copy()
    perturbed[index] += step
    diff = self._ctrv_step(perturbed, dt) - base
    jacobian[:, index] = diff / step
```

2.4 Process noise

The parameters q_x , q_y , q_v , q_ϕ , and $q_{\dot{\phi}}$ are the configured noise scales for the two position coordinates, speed, direction, and turn rate. The diagonal process-noise model is

$$\mathbf{Q} = \text{diag}\left((q_x \Delta t)^2, (q_y \Delta t)^2, (q_v \sqrt{\Delta t})^2, (q_\phi \sqrt{\Delta t})^2, (q_{\dot{\phi}} \sqrt{\Delta t})^2\right). \quad (27)$$

```
return np.diag([
    (self.config.process_noise_px * dt) ** 2,
    (self.config.process_noise_py * dt) ** 2,
    (self.config.process_noise_v * np.sqrt(dt)) ** 2,
    (self.config.process_noise_phi * np.sqrt(dt)) ** 2,
    (self.config.process_noise_phi_dot * np.sqrt(dt)) ** 2,
])
```

2.5 Position measurement update

LiDAR supplies only the two-dimensional obstacle position. Here $\mathbf{z}$ is the measurement vector, h is the measurement function, and $\mathbf{H}$ maps the five-state vector to the two measured position components:

$$\begin{aligned} \mathbf{z} &= \begin{bmatrix} p_x^{\text{meas}} & p_y^{\text{meas}} \end{bmatrix}^T, \\ h(\mathbf{x}) &= \begin{bmatrix} p_x & p_y \end{bmatrix}^T. \end{aligned} \quad (28)$$

with

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}. \quad (29)$$

The update equations are

$$\boldsymbol{\nu} = \mathbf{z} - \mathbf{H}\mathbf{x}^-, \quad (30)$$

$$\mathbf{S} = \mathbf{H}\mathbf{P}^-\mathbf{H}^T + \mathbf{R}, \quad (31)$$

$$\mathbf{K} = \mathbf{P}^-\mathbf{H}^T\mathbf{S}^{-1}, \quad (32)$$

$$\mathbf{x}^+ = \mathbf{x}^- + \mathbf{K}\boldsymbol{\nu}. \quad (33)$$

The obstacle EKF uses the Joseph covariance update,

$$\mathbf{P}^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-(\mathbf{I} - \mathbf{K}\mathbf{H})^T + \mathbf{K}\mathbf{R}\mathbf{K}^T. \quad (34)$$

```
innovation = z - H @ self.state
innovation_covariance = H @ self.covariance @ H.T + R
kalman_gain = (
    self.covariance @ H.T
    @ np.linalg.inv(innovation_covariance)
)
self.state = self.state + kalman_gain @ innovation

identity = np.eye(self.STATE_SIZE)
correction = identity - kalman_gain @ H
self.covariance = (
    correction @ self.covariance @ correction.T
    + kalman_gain @ R @ kalman_gain.T
)
```

If the innovation covariance is singular, the implementation uses `np.linalg.pinv()`. It also symmetrises the covariance, bounds speed and turn rate, and resets sufficiently small speeds to zero.

2.6 Future position and trajectory

For a prediction horizon $\tau \geq 0$, the predicted obstacle position is the pair of position components extracted by the subscript $[\cdot]_{p_x, p_y}$:

$$\mathbf{p}(\tau) = [f(\mathbf{x}, \tau)]_{p_x, p_y}. \quad (35)$$

```
def get_predicted_position(self, t_horizon):
    predicted_state = self._ctrv_step(
        self.state, max(float(t_horizon), 0.0)
    )
    return predicted_state[0:2].copy()
```

3 APF obstacle geometry

In this chapter, $\mathbf{r}$ is the relative position from the obstacle centre to the vessel, while $\mathbf{l}$ and $\mathbf{w}$ are orthogonal unit vectors along the obstacle length and width. The observed principal-component dimensions $pc1$ and $pc2$ define the corresponding semi-axes a_0 and b_0 . The scalars a and b are the projections of $\mathbf{r}$ onto these axes, ρ is the dimensionless elliptical level, $\mathbf{g}$ is the outward gradient, and $\hat{\mathbf{d}}$ is its unit direction. The scalar s is the outer field scale, u is the normalised decay coordinate, f_{fade} is the smooth fade factor, k_{rep} is the repulsive gain, and $\mathbf{F}_{\text{rep}}$ is the resulting repulsive force. A hat denotes a unit vector, $\|\cdot\|$ denotes Euclidean norm, and $\cdot$ denotes the inner product.

3.1 Elliptical level set

The measured obstacle dimensions $pc1$ and $pc2$ are obtained from the first and second principal components of the clustered LiDAR points. The axial projections are

$$a = \mathbf{r} \cdot \mathbf{l}, \quad b = \mathbf{r} \cdot \mathbf{w}. \quad (36)$$

The APF semi-axes are defined directly by the observed dimensions:

$$a_0 = \frac{pc1}{2}, \quad b_0 = \frac{pc2}{2}. \quad (37)$$

Minimum PC dimensions are used only when an observation is invalid or unavailable. The elliptical level is

$$\rho = \sqrt{\left(\frac{a}{a_0}\right)^2 + \left(\frac{b}{b_0}\right)^2}. \quad (38)$$

The outward gradient and corresponding unit direction are

$$\mathbf{g} = \frac{a}{a_0^2} \mathbf{l} + \frac{b}{b_0^2} \mathbf{w}, \quad (39)$$

$$\hat{\mathbf{d}} = \frac{\mathbf{g}}{\|\mathbf{g}\|}. \quad (40)$$

The condition $\rho < 1$ indicates that the vessel has entered the observed obstacle boundary. The repulsive field is inactive when $\rho \geq s$.

```

pc1_m, pc2_m = self.obstacle_pc_dimensions(obstacle)
semi_length_m = 0.5 * pc1_m
semi_width_m = 0.5 * pc2_m

along = float(np.dot(offset, length_axis))
across = float(np.dot(offset, width_axis))
level = float(np.hypot(
    along / semi_length_m,
    across / semi_width_m,
))
gradient = (
    along / (semi_length_m * semi_length_m) * length_axis
    + across / (semi_width_m * semi_width_m) * width_axis
)
away = gradient / np.linalg.norm(gradient)

```

3.2 Smooth elliptical repulsion

For $s > 1$, the implementation applies a cubic smoothstep decay. The variable u clips the level interval $[1, s]$ to $[0, 1]$, so that the fade remains bounded both inside the obstacle and outside the influence region:

$$u = \text{clip}\left(\frac{\rho - 1}{s - 1}, 0, 1\right), \quad (41)$$

$$f_{\text{fade}} = 1 - (3u^2 - 2u^3), \quad (42)$$

$$\mathbf{F}_{\text{rep}} = k_{\text{rep}} f_{\text{fade}} \hat{\mathbf{d}}. \quad (43)$$

The force is maximal at and inside $\rho = 1$, and it decreases smoothly to zero at $\rho = s$.

```

u = np.clip(
    (level - 1.0) / (outer_scale - 1.0),
    0.0,
    1.0,
)
fade = 1.0 - (3.0 * u * u - 2.0 * u * u * u)
return gain * fade * np.asarray(away).reshape(2), True

```

The project uses two fields with this form. The first is centred on the current LiDAR obstacle, while the second is centred on the EKF/CPA-predicted obstacle. Their outer scales are selected from encounter-specific profiles: head-on, overtaking, crossing, and static-obstacle cases. Predicted fields use a separate profile table from current-observation fields.

3.3 Observed dimensions in snapshot generation

The offline potential-field renderer uses the same observed $pc1$ and $pc2$ dimensions. The stored equivalent radius is not used to reconstruct the obstacle dimensions when these observations are present; fallback values are the configured minimum PC dimensions.

```

pc1_m = positive(obstacle.get("pc1_m"), minimum_pc1_m)
pc2_m = positive(obstacle.get("pc2_m"), minimum_pc2_m)
return max(pc1_m, minimum_pc1_m), max(
    min(pc2_m, pc1_m), minimum_pc2_m
)

```

4 APF attractive terms

In this chapter, all attractive-force vectors are expressed in the vessel body frame. The target vector is $\mathbf{t}$, its distance is d , k_g is the goal gain, and d_{sat} is the maximum distance used for saturating the goal-force magnitude. The path variables $\mathbf{s}$, $\mathbf{g}$, $\mathbf{p}$, $\mathbf{d}_{\text{path}}$, L_{path} , λ , and $\mathbf{p}_{\text{path}}$ denote the start, goal, current position, route vector, route length, clipped projection coefficient, and selected route point. The transformation $T_{\text{earth} \rightarrow \text{body}}$ maps earth-frame vectors into body-frame vectors. The path error and its norm are $\mathbf{e}_{\text{path}}$ and d_{path} ; k_p is the path gain, and $d_{\text{threshold}}$ is the path dead-band threshold. Finally, $\mathbf{F}_{\text{goal}}$, $\mathbf{F}_{\text{path}}$, and $\mathbf{F}_{\text{att}}$ denote the goal, path, and total attractive forces.

4.1 Goal attraction

Let $\mathbf{t}$ be the target position in the body frame, with $d = \|\mathbf{t}\|$. The saturated goal-attraction term is

$$\mathbf{F}_{\text{goal}} = k_g \min(d, d_{\text{sat}}) \frac{\mathbf{t}}{d}. \quad (44)$$

```
distance_m = float(np.linalg.norm(target_body))
magnitude = self.apf_goal_gain * min(
    distance_m,
    self.apf_attraction_saturation_m,
)
goal_force = magnitude * target_body / distance_m
```

4.2 Path attraction

The path-attraction field keeps the vessel close to the straight route from the initial position to the goal. Let $\mathbf{s}$ and $\mathbf{g}$ denote the start and goal positions in the north-east frame, and let $\mathbf{p}$ be the current vessel position. The route vector and its squared length are

$$\begin{aligned} \mathbf{d}_{\text{path}} &= \mathbf{g} - \mathbf{s}, \\ L_{\text{path}}^2 &= \mathbf{d}_{\text{path}}^\top \mathbf{d}_{\text{path}}. \end{aligned} \quad (45)$$

If L_{path}^2 is smaller than a numerical tolerance, the start and goal are treated as coincident and the path-attraction force is set to zero.

For a valid route, the projection coefficient of the current position onto the start-to-goal segment is

$$\lambda^* = \frac{(\mathbf{p} - \mathbf{s})^\top \mathbf{d}_{\text{path}}}{L_{\text{path}}^2}. \quad (46)$$

The implementation clips this coefficient to $[0, 1]$:

$$\lambda = \text{clip}(\lambda^*, 0, 1). \quad (47)$$

Consequently, the reference point is on the finite route segment rather than on its infinite extension,

$$\mathbf{p}_{\text{path}} = \mathbf{s} + \lambda \mathbf{d}_{\text{path}}. \quad (48)$$

When the perpendicular projection falls before the start or beyond the goal, the start or goal endpoint is therefore used as the reference point.

The vector from the vessel to this reference point is first calculated in the north-east frame and then transformed into the vessel body frame. With $T_{\text{earth} \rightarrow \text{body}}$ denoting the rotation from the earth frame to the body frame,

$$\mathbf{e}_{\text{path}} = T_{\text{earth} \rightarrow \text{body}}(\mathbf{p}_{\text{path}} - \mathbf{p}). \quad (49)$$

The norm $d_{\text{path}} = \|\mathbf{e}_{\text{path}}\|$ is the current cross-track distance to the selected route point. The resulting path force is

$$\mathbf{F}_{\text{path}} = \begin{cases} \mathbf{0}, & d_{\text{path}} < d_{\text{threshold}}, \\ k_p \mathbf{e}_{\text{path}}, & d_{\text{path}} \geq d_{\text{threshold}}. \end{cases} \quad (50)$$

The dead-band threshold prevents small position-estimation fluctuations from causing continuous corrective commands while the vessel is already close to the route. Outside this band, the force magnitude is proportional to the cross-track error and its direction points toward the route.

```
path_vec = self.goal_ne - self.start_ne
path_len_sq = float(np.dot(path_vec, path_vec))
if path_len_sq < 1e-9:
    return np.zeros(2, dtype=float)

current_ne = np.array(
    [float(self.North), float(self.East)], dtype=float
)
ratio = float(np.clip(
    np.dot(current_ne - self.start_ne, path_vec)
    / path_len_sq,
    0.0,
    1.0,
))
closest_ne = self.start_ne + ratio * path_vec
to_path_body = self.earth_point_to_body(closest_ne)
distance_m = float(np.linalg.norm(to_path_body))

if distance_m < self.apf_path_threshold_m or distance_m < 1e-6:
    return np.zeros(2, dtype=float)
return self.apf_path_gain * to_path_body
```

The total attractive term is the vector sum of the two components:

$$\mathbf{F}_{\text{att}} = \mathbf{F}_{\text{goal}} + \mathbf{F}_{\text{path}}. \quad (51)$$

5 CPA-based dynamic-obstacle risk

In this chapter, $\mathbf{p}$ is the obstacle position relative to the vessel, $\mathbf{v}_{\text{obs}}$ and $\mathbf{v}_{\text{own}}$ are the obstacle and own- vessel velocities, and $\mathbf{v}_{\text{rel}}$ is their relative velocity. TCPA is the non-negative time to the closest point of approach, and DCPA is the distance at that time. The symbols $\mathbf{p}(t)$ and $\mathbf{v}_{\text{rel}}$ describe relative position and velocity, while $\|\cdot\|$ denotes vector magnitude. The zero-speed threshold is a numerical guard used when the relative-velocity norm is too small.

The relative velocity is

$$\mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{obs}} - \mathbf{v}_{\text{own}}. \quad (52)$$

If $\|\mathbf{v}_{\text{rel}}\|^2$ is sufficiently small, the implementation returns

$$\text{TCPA} = \infty, \quad \text{DCPA} = \|\mathbf{p}\|. \quad (53)$$

Otherwise,

$$\text{TCPA} = \max\left(-\frac{\mathbf{p} \cdot \mathbf{v}_{\text{rel}}}{\|\mathbf{v}_{\text{rel}}\|^2}, 0\right), \quad (54)$$

$$\text{DCPA} = \|\mathbf{p} + \mathbf{v}_{\text{rel}}\text{TCPA}\|. \quad (55)$$

```
rel_vel = obs_vel_body - own_vel_body
rel_speed_sq = float(np.dot(rel_vel, rel_vel))

if rel_speed_sq < 1e-9:
    return np.inf, float(np.linalg.norm(obs_pos_body))

tcpa = max(
    -float(np.dot(obs_pos_body, rel_vel)) / rel_speed_sq,
    0.0,
)
dcpa = float(np.linalg.norm(
    obs_pos_body + rel_vel * tcpa
))
return tcpa, dcpa
```

When the predicted risk is active, the controller creates a virtual elliptical field at the future collision location. This field provides an earlier avoidance response than the current-obstacle field alone.

6 APF force composition and control output

In this chapter, $\mathbf{F}$ is the total body-frame guidance force, $\mathbf{F}_{\text{att}}$ is the attractive force, and $\mathbf{F}_{\text{rep}}$ is the repulsive force. The scalar ρ_{nearest} is the smallest active elliptical level, α_{min} is the lower bound on the attractive-force weight, and α is the resulting blending coefficient. In the steering equations, F_x and F_y are the forward and lateral components of $\mathbf{F}$, θ_F is its steering angle, k_h is the heading gain, ω_c is the commanded yaw rate, and ω_{max} is its magnitude limit. The operator `atan2` returns the signed angle from the two force components, while `clip` limits a scalar to a prescribed interval. The final command contains surge speed v_c and yaw rate ω_c .

6.1 Total force

When no repulsive field is active, the total force equals the attractive force:

$$\mathbf{F} = \mathbf{F}_{\text{att}}. \quad (56)$$

When repulsion is active,

$$\mathbf{F} = \mathbf{F}_{\text{rep}} + \alpha \mathbf{F}_{\text{att}}, \quad (57)$$

When repulsion is active, α is computed from the nearest active field, where

$$\alpha = \text{clip}(\rho_{\text{nearest}}, \alpha_{\text{min}}, 1). \quad (58)$$

As the vessel approaches an obstacle, the attractive contribution decreases and collision avoidance receives priority.

```
force_body = attractive_force + repulsive_force
```

```

if any_repulsion and np.isfinite(nearest_active_level):
    alpha = np.clip(
        nearest_active_level,
        self.apf_close_quarters_force_scale,
        1.0,
    )
    force_body = repulsive_force + alpha * attractive_force

```

The controller also preserves the COLREGs-selected avoidance side. If the forward force becomes non-positive, it retains a lateral component and inserts a small positive forward component.

6.2 Force direction and yaw-rate command

The steering angle is

$$\theta_F = \text{atan2}(F_y, F_x). \quad (59)$$

After limiting this angle, the yaw-rate command is

$$\omega_c = \text{clip}\left(-\frac{k_h \theta_F}{\Delta t}, -\omega_{\max}, \omega_{\max}\right). \quad (60)$$

```

force_angle = np.arctan2(
    steering_force[1], steering_force[0]
)
force_angle = np.clip(
    force_angle,
    -self.apf_heading_step_limit_rad,
    self.apf_heading_step_limit_rad,
)
u_cmd[1, 0] = np.clip(
    -self.apf_heading_gain * force_angle
    / max(self.lastdt, 1e-3),
    -self.w_max,
    self.w_max,
)

```

During APF avoidance, the force magnitude does not directly determine surge speed. The encounter type and COLREG state determine the surge command:

```

surge_cmd = self.apf_encounter_speed_m_s(
    self.apf_encounter_mode,
    nearest_active_level,
    force_angle,
)
u_cmd[0, 0] = np.clip(surge_cmd, 0.0, self.v_max)

```

The final APF control output is

$$\mathbf{u}_{\text{APF}} = \begin{bmatrix} v_c & \omega_c \end{bmatrix}^T. \quad (61)$$

7 Processing chain

The complete processing chain is:

1. cluster the LiDAR point cloud using DBSCAN and estimate obstacle dimensions using PCA;
2. associate detections with obstacle tracks;
3. predict and correct each obstacle state with the CTRV-EKF;
4. compute obstacle velocity, TCPA, DCPA, and future positions;
5. construct the current and EKF/CPA-predicted elliptical fields;
6. combine APF attraction, repulsion, and the COLREGs side constraint;
7. convert the resultant-force direction into surge and yaw-rate commands; and
8. allocate the commanded force and moment to the two thrusters.

The vessel EKF supplies the vessel position, heading, and velocity at the start of this chain. The obstacle EKF supplies the predicted dynamic-obstacle state used by the APF.

8 Implementation index

Function	Source-code entry point
Vessel motion model	<code>motion_model</code>
Vessel EKF prediction	<code>extended_kalman_filter_predict</code>
Vessel EKF update	<code>extended_kalman_filter_update</code>
Pose measurement model	<code>h_pose_update</code>
Yaw-rate measurement model	<code>h_grate_update</code>
Obstacle CTRV propagation	<code>ObstacleEKF._ctrv_step</code>
Obstacle EKF prediction and update	<code>ObstacleEKF.predict, ObstacleEKF.update</code>
Elliptical geometry	<code>ellipse_level_and_away</code>
Smooth elliptical repulsion	<code>smooth_ellipse_repulsion</code>
Goal and path attraction	<code>apf_goal_attraction_body,</code> <code>apf_path_attraction_body</code>
CPA computation	<code>apf_cpa_metrics</code>
Obstacle repulsion composition	<code>apf_repulsion_for_obstacle</code>
APF control output	<code>compute_apf_control</code>